

CS550 PA#1 | Project Report

A Simple Napster-Style Peer-to-Peer File Sharing System

Documentation prepared September 28, 2026 from the supplied source and recorded results.

1. Completion statement

The implementation provides a central metadata index, exact filename lookup, direct peer downloads, threaded request handling and configurable registration-time replication. The main programs use Python sockets and threads. Text and binary files use the same raw-byte transfer protocol. A local verification of factor-2 replication and binary download passed during this review.

The evaluation is partially complete. Search and small-file transfer records contain three successful trials per configuration. Medium-file experiments use 800 transfers instead of 1,000. Large-file experiments use the required 8 transfers, but only two paired concurrent trials succeeded. The last concurrent trial recorded three failures on peer_a. Reduced staged generation does not establish the required simultaneous dataset.

2. Deployment and architecture

Role	Recorded endpoint
VM3: central index	10.0.2.6:9000
VM1: peer_a	10.0.2.5:9001
VM2: peer_b	10.0.2.15:9002

The index keeps filename-to-owner metadata in memory; file contents remain on peers. A downloader queries the index and then connects directly to the selected peer. Each peer can serve and request files concurrently. SSH coordinates experiments between VM1 and VM2; it is not the application file-transfer protocol.

The assignment target is Ubuntu 26.04 LTS on three VMs. The result metadata supplies endpoint information but not a complete OS, CPU, RAM and hypervisor inventory. Add verified deployment details and actual VM screenshots before final submission. No hardware values or screenshots have been invented.

Implementation and correctness

3. Index, transport and synchronization

server.py implements registry, search and list_peers over newline-delimited JSON. file_index maps a name to owners; peer_registry stores endpoint records. A threading.Lock protects both structures. Each connection is handled by a daemon thread. Search excludes the requester when its peer ID is supplied and returns all remaining exact matches.

peer_server.py implements obtain and replicate. An obtain reply sends a JSON header containing file_size, then streams that number of bytes in 64 KiB chunks. download.py saves to a unique temporary .part file and atomically replaces the destination only on completion. The downloader registers its new ownership afterward. Filenames are checked and serving symlinks is rejected.

4. Replication

The index supplies a system-wide replication factor, interpreted as the total copies including the source. register.py registers names in batches, checks owners and asks other registered peers to pull missing replicas. Destinations register completed replicas. Replica pulls are serialized per destination with a lock. There is no background re-replication or failure detector; re-running registration is required to retry an under-replicated file.

The latest staged environment reports factor 2, but its worker directly registers files without enforcing replication. Therefore the benchmark dataset is not demonstrated to have two copies per file. The historical index_config.json says factor 1, but server.py does not read that file; command-line flags determine the running value.

5. Functional verification and output

A fresh local loopback test launched the actual index and two peer servers, registered an empty peer, registered a 65,536-byte binary source at factor 2, verified the replica byte for byte, queried both owners, downloaded from an explicitly selected source, and checked a missing-name search. The source and downloaded bytes matched. This is functional evidence only, not a three-VM performance result.

The complete captured transcript is docs/local_smoke_output.txt and output.txt. It contains the required display file message and formatted lookup output. SHA-256 of the tested binary:
7daca2095d0438260fa849183dfc67faa459fdf4936e1bc91eec6b281b27e4c2.

Local Python source compilation passed. These checks do not certify the full dataset, sustained concurrency, target-VM OS version or completion of the failed experiment.

Performance methods and results

6. Measurement method

Units are binary: 1 KiB = 1,024 bytes, 1 MiB = 1,048,576 bytes, 1 GiB = 1,073,741,824 bytes. Weak scaling uses 10,000 sequential requests per active requester: 10,000 total with one requester and 20,000 total with two. Both peer services remain available in one-requester tests. Two-requester completion time is the latest finish minus the earliest start.

Strong scaling holds total transfer work fixed across requester counts. Small-file tests use 10,000 total files. The staged medium tests use 800, and the large tests use 8. Small-file timing includes search, transfer and size checking; staged timing additionally includes SHA-256 checking and deletion of each received copy. Registration is outside timed intervals. Caches are not cleared and file data is not fsync-ed. Do not interpret cross-size differences as pure network bandwidth.

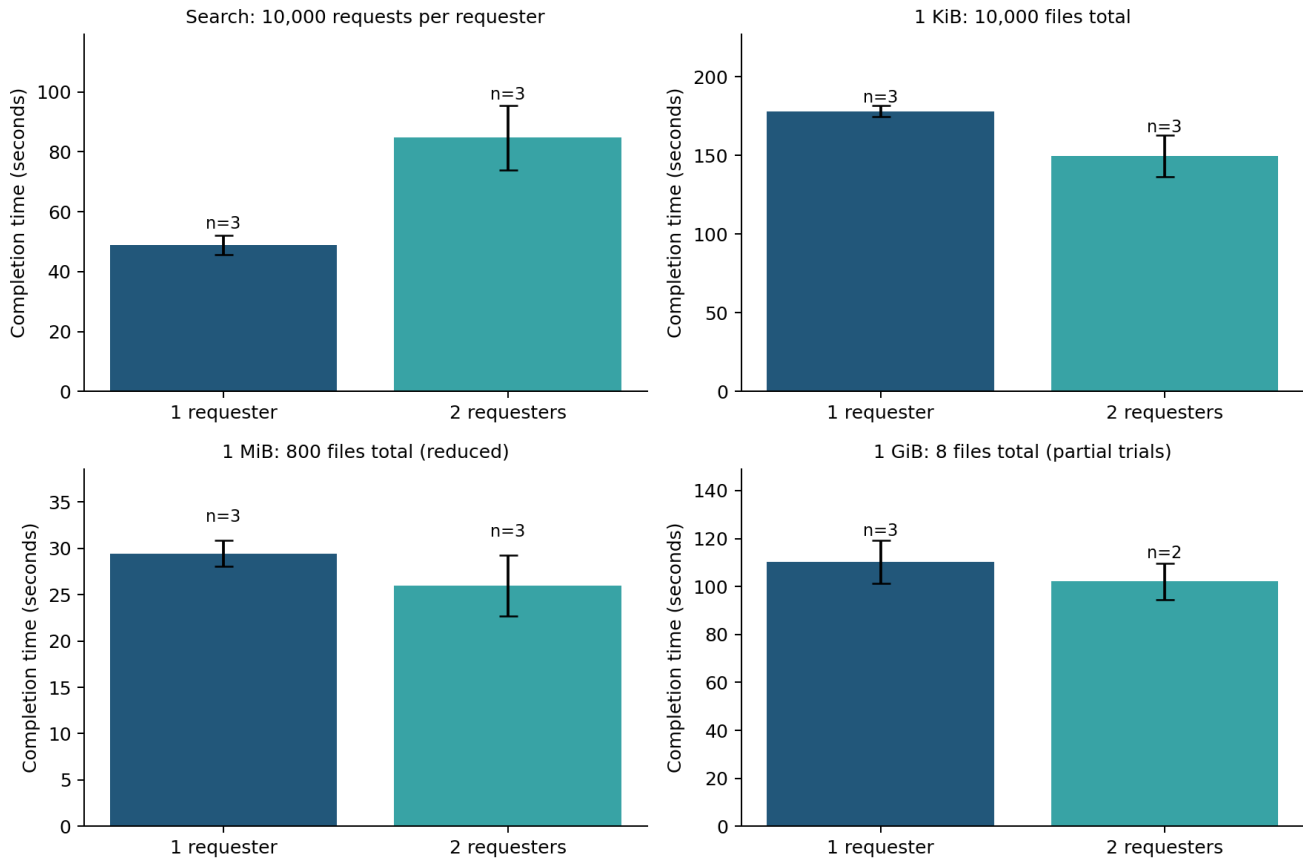
Mean is the arithmetic mean of trial completion times. Error bars and the table use sample standard deviation (denominator n-1), not per-request latency variation. Only successful completed paired trials enter the averages. Search start skews for the three paired trials are 0.517s, 0.014s, 0.018s. Cross-VM wall-clock timing assumes synchronized clocks.

Case	Requesters	Work total	n	Mean s	SD s
Search	1	10000	3	48.942	3.297
Search	2	20000	3	84.696	10.697
1 KiB transfer	1	10000	3	178.064	3.485
1 KiB transfer	2	10000	3	149.651	13.285
1 MiB reduced	1	800	3	29.443	1.413
1 MiB reduced	2	800	3	25.983	3.306
1 GiB partial	1	8	3	110.214	8.981
1 GiB partial	2	8	2	102.180	7.604

Medium rows are reduced-workload results. Large two-requester statistics use only two successful trials and are provisional. The unsuccessful third trial is disclosed separately; it is not converted into a successful completion time.

Performance figures and interpretation

Recorded performance: mean $\pm$ sample standard deviation



7. Does the system scale to two requesters?

Search time rises from 48.94s to 84.70s when total work doubles. Weak-scaling efficiency T1/T2 is 57.8%; aggregate throughput improves by about 1.16x, not the ideal 2x. This suggests limited scalability under concurrent index load; the data does not isolate the lock, CPU, network or host scheduling as the cause.

Small-file strong-scaling speedup is 1.19x. The reduced medium-file study shows 1.13x and the partial large-file study shows 1.08x. All are below ideal 2x. Medium and large results do not show strong benefits in these runs, and large-file variation plus the failed trial weaken that conclusion. Additional successful, requirement-matching trials are needed.

Possible contributors include repeated TCP setup, index serialization, shared host CPU/storage, bidirectional network contention, caching and SHA-256 work. These are hypotheses, not measured bottleneck diagnoses. Comparisons across sizes also differ in verification and cleanup work.

Scalability, limitations and evidence

8. Extrapolation to 1,000 and one billion peers

At 1,000 peers, a single in-memory index must hold more ownership records and serve many more connections. One thread per connection increases scheduling and memory cost; the global lock limits concurrent metadata updates/lookups. Direct peer data transfers avoid sending file bytes through the index, but the index remains a bottleneck and single point of failure. Small-file workloads are more sensitive to lookup/setup overhead, while larger transfers can be limited by network and disk capacity. Two-peer measurements cannot quantify the 1,000-peer throughput.

At one billion peers, this implementation is not practical. Central metadata, connections, failure handling and a single failure domain exceed its design. A different architecture would need partitioned/distributed indexes, bounded asynchronous or pooled handling, replicated metadata services, peer liveness management and recovery. This is a design inference, not an experimentally validated extrapolation.

9. Known failures and assignment deviations

The latest third two-requester 1 GiB trial recorded one successful transfer and three failures on peer_a: one timeout and two No route to host errors. Root cause is not established by the logs. There is no complete paired third-trial result in this archive. Earlier staged attempts also stopped for connectivity/capacity reasons; they are retained as provenance and not mixed into the selected latest results.

The handout requires 1M small, 1K medium and 10 large source files per peer together in one directory, plus a medium transfer study of 1K files. The staged method instead uses 800 medium and 8 large sources, one size at a time, deleting generated files. The count of 8 large transfers is correct; it must not be confused with the required 10-file resident large dataset. Full simultaneous dataset and replica coverage remain unverified.

10. Evidence locations

Search: results/pilot_1peer_* and results/pilot_2peers_* (both peer_a and recovered peer_b records). Small transfers: results/pilot_transfer_1peer_* and results/auto_20260928T041658_762595Z/. Medium and large: results/staged_20260928T151557_309964Z/. Derived table: docs/performance_summary.csv. Functional transcript: docs/local_smoke_output.txt. Requirement source: cs550_pa1.docx, pages 1-3, supplied assignment document.

Before final submission, add original VM screenshots and the complete per-student hourly development log including all AI prompts. Review student identity, target-VM environment and current Canvas deadline. See SUBMISSION_CHECKLIST.txt. This documentation does not claim Canvas upload or private git submission has occurred.